

Lesson 2 — Methods & Functions



RamRodz Robotics · Team 6366 · Programming Worksheet

NAME: _____ DATE: _____

RAM RODZ ROBOTICS | TEAM #6366 | HILLCREST & WOODMONT HIGH SCHOOLS ·
SIMPSONVILLE, SC

Focus Check-In:

Struggling

Okay

Ready

TODAY'S BIG QUESTION

The robot has to aim, spin up the shooter, fire, and stop — all in the right order, every match. How do you write that sequence once and reuse it anywhere without copying the same 10 lines of code everywhere?

Parking Lot — Random Thoughts

Had a random thought? Question pop into your head? Park it here — we'll come back to it.

1 Key Words — Check Off When Explained



Click each card when the instructor explains that term. All 6 checked = section done

Method

A named block of code that performs one specific task — write it once, run it anywhere by calling its name.

Parameter

A variable in the method's () that receives a value when the method is called — a placeholder for input.

Argument

The actual value passed when calling a method. In `setSpeed(0.8)`, the argument is 0.8.

void

The return type that means "this method does a job but doesn't send anything back."

return

The keyword that sends a value back to whatever code called the method — ends the method immediately.

Call / Invoke

To run a method by writing its name followed by () — like pressing a labeled button that triggers a sub-routine.

2 What Is a Method?



A method is like a labeled sub-routine on the robot. You give a block of steps a name, and then anywhere you need those steps you just use the name — one short line instead of repeating all the code.

☐ Listen: Instructor explains why copy-pasting code ten times causes problems

Java — Basic Method (no parameters, returns nothing)

```
// Define the method — write this once, anywhere in your robot file
void fireShooter() {
    shooterSpeed = 1.0;    // set shooter to 100% power
    isRunning      = true;  // mark it as active
}
```

Plain English: **void** means "nothing comes back out." The name **fireShooter** is what you'll type to run it. The curly braces { } hold every step the method performs.

ANATOMY OF A METHOD

```
void  fireShooter  ( )  { ... }
```

void — return type (nothing returned)

fireShooter — the method's name

() — parameters go here (empty = none)

{ } — the body — your steps go here

- ☐ Observe: I can point to the 4 parts of the method above — return type, name, params, body

Quick Question — Fill In

- ☐ Done: I filled in both answers above

3 Calling a Method — Making It Run



- ☐ Listen: Instructor explains the difference between defining a method and calling it

Java — Defining vs. Calling

```
// ① DEFINE — write it once
void fireShooter() {
    shooterSpeed = 1.0;
    isRunning    = true;
}

// ② CALL — use it whenever you need it
if (buttonPressed) {
    fireShooter(); // runs every line inside the method
}
```

💡 Plain English: Defining a method is like writing a recipe. Calling it is like actually cooking the meal. You write the recipe once — cook it as many times as you want.

☐ Observe: I see that fireShooter() in the if-block runs all the code inside the method definition

Java — Same Method Called in Two Different Places

```
// In teleop (driver control)
if (buttonPressed) {
    fireShooter();    // same method — no copy-paste needed
}

// In auto (no driver)
if (autoMode) {
    fireShooter();    // called again from a completely different spot
}
```

💡 Plain English: One definition, used in two places. If you ever need to change how the shooter fires, you update **one method** and it's fixed everywhere automatically.

Try It — Write the Call

☐ Done: I wrote the method call above

4 Parameters — Giving a Method Information



💡 A method with no parameters always does the exact same thing. What if you want to set the drive speed to *any* value? Add a **parameter** — a slot the caller fills in with a specific value each time.

☐ Listen: Instructor explains parameters vs. arguments and why they're different words

Java — Method with One Parameter

```
// Parameter: "double speed" — expects a decimal number, calls it 'speed'
void setDriveSpeed(double speed) {
    driveSpeed = speed;    // use whatever was passed in
}

// Call it with different arguments each time
setDriveSpeed(0.8);    // 80% power — speed becomes 0.8 inside
setDriveSpeed(0.5);    // 50% power — speed becomes 0.5 inside
setDriveSpeed(0.0);    // stop — speed becomes 0.0 inside
```

💡 Plain English: **double speed** is the *parameter* — a labeled slot waiting to be filled. The **0.8** in `setDriveSpeed(0.8)` is the *argument* — the actual value dropped into that slot for that call.

☐ Observe: I can trace how the argument 0.8 travels into the parameter "speed" and becomes driveSpeed

Java — Method with Two Parameters

```
// Two parameters — left and right sides of a tank drive
void setTankDrive(double leftSpeed, double rightSpeed) {
    driveLeft = leftSpeed;    // left wheels
    driveRight = rightSpeed;  // right wheels
}

// Spin left: left backward, right forward
setTankDrive(-0.5, 0.5);

// Drive straight forward at full speed
setTankDrive(1.0, 1.0);
```

💡 Plain English: Separate multiple parameters with commas — same in the definition and in the call. Order matters: first argument → first parameter, second argument → second parameter.

Try It — Parameters

☐

Done: I answered all three parameter questions above

5 Return Values — Getting Information Back



 Sometimes you don't want a method to *do* something — you want it to *check* something and hand back an answer. Replace **void** with the type you're sending back, and use **return** to send it.

☐

Listen: Instructor explains when you'd use a return value instead of void

Java — Method That Returns a boolean

```
// Ask: is the robot going fast? Returns true or false
boolean isAtFullSpeed() {
    return driveSpeed > 0.9;    // true if above 90%, false otherwise
}

// Use the answer directly in an if-statement
if (isAtFullSpeed()) {
    warningLight = true;    // turn on warning light at full speed
}
```

 Plain English: Swap **void** for **boolean** to say "this method answers yes or no." The **return** line is where the answer gets sent back — and the method stops right there.

☐

Observe: I see that the return type (boolean) must match what's after the return keyword

Java — Method That Returns a double (with safety check)

```
// Clamp the throttle so it never exceeds 1.0
double getSafeSpeed(double rawThrottle) {
    if (rawThrottle > 1.0) {
        return 1.0;      // cap at max — exit the method here
    }
    return rawThrottle;  // normal case — pass the value through
}

// Store the safe value in driveSpeed
driveSpeed = getSafeSpeed(throttle);
```

💡 Plain English: You can have more than one **return** — the method exits at the first one it reaches. Check the dangerous case first and return early; let the normal case fall through to the end.

Quick Question — Return Values

☐ Done: I answered all three return-value questions above

Robot Scenario — Put It All Together

Autonomous Shooting Routine

During autonomous, your robot needs to check if the turret is aimed before firing. Your job is to write two methods and wire them together:

- **isAimed()** — returns true if turretAngle is between 43 and 47 degrees
- **fireShooter()** — sets shooterSpeed to 1.0 and isRunning to true
- An **if-statement** that calls isAimed() and, if true, calls fireShooter()



Listen: Instructor walks through the scenario before writing anything



Step 1 — Write the isAimed() method



Step 2 — Write the fireShooter() method



Step 3 — Wire them together with an if-statement



Done: I completed all 3 steps of the robot scenario

 End of Lesson — I Can...

- ☐ Explain what a method is in plain English — without using the word "method"
- ☐ Write a void method with no parameters that sets a robot variable
- ☐ Write a method that accepts a parameter and uses it inside the body
- ☐ Write a method that returns a boolean or double value using the return keyword
- ☐ Call a method inside an if-statement in my robot code